

Expressions and Assignment Statements

Introduction

- Expressions are the fundamental means of specifying computations in a programming language
- To understand expression evaluation, need to be familiar with the orders of operator and operand evaluation
- Essence of imperative languages is dominant role of assignment statements

Arithmetic Expressions

- Arithmetic evaluation was one of the motivations for the development of the first programming languages
- Arithmetic expressions consist of operators, operands, parentheses, and function calls
- In most languages, binary operators are infix, except in Scheme and LISP, in which they are prefix; Perl also has some prefix binary iterators
- Most unary operators are prefix, but the ++ and -- operators in C-based languages can be either prefix or postfix

Arithmetic Expressions: Design Issues

- Design issues for arithmetic expressions
 - Operator precedence rules?
 - Operator associativity rules?
 - Order of operand evaluation?
 - Operand evaluation side effects?
 - Operator overloading?
 - Type mixing in expressions?

Arithmetic Expressions: Operators

- A **unary** operator has one operand
- A **binary** operator has two operands
- A **ternary** operator has three operands

Arithmetic Expressions: Operator Precedence Rules

- The *operator precedence rules* for expression evaluation define the order in which “adjacent” operators of different precedence levels are evaluated
- Typical precedence levels
 - parentheses
 - unary operators
 - ** (if the language supports it)
 - *, /
 - +, -

Arithmetic Expressions: Operator Associativity Rule

- The **operator associativity rules** for expression evaluation define the order in which adjacent operators with the same precedence level are evaluated
- Typical associativity rules
 - Left to right, except **, which is right to left
 - Sometimes unary operators associate right to left (e.g., in FORTRAN)
- APL is different; all operators have equal precedence and all operators associate right to left
- Precedence and associativity rules can be overridden with parentheses

Associativity Examples

- Fortran
 - Left: *, /, +, -
 - Right: **
- c-based
 - Left: *, /, %, binary +, binary -
 - Right: ++, --, unary -, unary +
- Ada
 - Left: all except **
 - Nonassociative: **
- APL
 - Right: all

Expressions in Ruby and Scheme

- All arithmetic, relational, and assignment operators, as well as array indexing, shifts, and bit-wise logic operators, are implemented as methods
 - One result of this is that these operators can all be overridden by application programs
- Scheme (and Common Lisp)
 - All arithmetic and logic operations are by explicitly called subprograms
 - $a + b * c$ is coded as `(+ a (* b c))`

Arithmetic Expressions: Conditional Expressions

- Conditional Expressions
 - C-based languages (e.g., C, C++)
 - An example:

average = (count == 0)? 0 : sum / count

- Evaluates as if written like

if (count == 0)

average = 0

else

average = sum / count

Arithmetic Expressions: Operand Evaluation Order

- *Operand evaluation order*
 - Variables: fetch the value from memory
 - Constants: sometimes a fetch from memory; sometimes the constant is in the machine language instruction
 - Parenthesized expressions: evaluate all operands and operators first
 - The most interesting case is when an operand is a function call

Arithmetic Expressions: Potentials for Side Effects

- ***Functional side effects***: when a function changes a two-way parameter or a non-local variable
- Problem with functional side effects:
 - When a function referenced in an expression alters another operand of the expression; e.g., for a parameter change:

`a = 10;`

`/* assume that fun changes its parameter */`

`b = a + fun(&a);`

Functional Side Effects

- Two possible solutions to the problem
 - Write the language definition to disallow functional side effects
 - No two-way parameters in functions
 - No non-local references in functions
 - **Advantage:** it works!
 - Used by Ada
 - **Disadvantage:** inflexibility of one-way parameters and lack of non-local references
 - Write the language definition to demand that operand evaluation order be fixed
 - **Disadvantage:** limits some compiler optimizations
 - Java requires that operands appear to be evaluated in left-to-right order

Referential Transparency

- A program has the property of *referential transparency* if any two expressions in the program that have the same value can be substituted for one another anywhere in the program, without affecting the action of the program

```
result1 = (fun(a) + b) / (fun(a) - c);  
temp = fun(a);  
result2 = (temp + b) / (temp - c);
```

If fun has no side effects, `result1 = result2`

Otherwise, not, and referential transparency is violated

Referential Transparency

(continued)

- Advantage of referential transparency
 - Semantics of a program is much easier to understand if it has referential transparency
- Because they do not have variables, programs in pure functional languages are referentially transparent
 - Functions cannot have state, which would be stored in local variables
 - If a function uses an outside value, it must be a constant (there are no variables). So, the value of a function depends only on its parameters

Overloaded Operators

- Use of an operator for more than one purpose is called *operator overloading*
- Some are common (e.g., + for int and float)
- Some are potential trouble (e.g., * in C and C++)
 - Loss of compiler error detection (omission of an operand should be a detectable error)
- Some loss of readability

Overloaded Operators

- C++, C#, F#, and Ada allow user-defined overloaded operators
- When sensibly used, such operators can be an aid to readability (avoid method calls, expressions appear natural)
- Potential problems:
 - Users can define nonsense operations
 - Readability may suffer, even when the operators make sense

Type Conversions

- A *narrowing conversion* is one that converts an object to a type that cannot include all of the values of the original type e.g., float to int
- A *widening conversion* is one in which an object is converted to a type that can include at least approximations to all of the values of the original type e.g., int to float

Type Conversions:

Mixed Mode

- A *mixed-mode expression* is one that has operands of different types
- A *coercion* is an implicit type conversion
- Disadvantage of coercions:
 - They decrease the type error detection ability of the compiler
- In most languages, all numeric types are coerced in expressions, using widening conversions
- In ML, F#, and Ada, there are no coercions in expressions

Explicit Type Conversions

- Called *casting* in C-based languages
- Examples
 - C: (int)angle
 - Ada, F#: float (Sum)

Note that Ada's syntax is similar to that of function calls

Errors in Expressions

- Causes
 - Inherent limitations of arithmetic
 - e.g., division by zero
 - Limitations of computer arithmetic
 - e.g. overflow
- Often ignored by the run-time system

Relational and Boolean Expressions

- Relational Expressions
 - Use relational operators and operands of various types
 - Evaluate to some Boolean representation
 - Operator symbols used vary somewhat among languages (`!=`, `/=`, `~=`, `.NE.`, `<>`, `#`)
- JavaScript and PHP have two additional relational operator, `===` and `!==`
 - Similar to their cousins, `==` and `!=`, except that they do not coerce their operands
 - Ruby uses `==` for equality relation operator that uses coercions and `eq?` for those that do not

Relational and Boolean Expressions

- Boolean Expressions
 - Operands are Boolean and the result is Boolean
 - Example operators

FORTRAN 77

.AND.

.OR.

.NOT.

FORTRAN 90

and

or

not

C

&&

||

!

Ada

and

or

not

xor

Relational and Boolean Expressions

- Boolean Expressions
 - Operands are Boolean and the result is Boolean
- C89 has no Boolean type--it uses int type with 0 for false and nonzero for true
- One odd characteristic of C's expressions:
 - $a < b < c$ is a legal expression, but the result is not what you might expect:
 - Left operator is evaluated, producing 0 or 1
 - The evaluation result is then compared with the third operand (i.e., c)

Relational and Boolean Expressions

- Precedence of all Ada operators:
******, **abs**, **not**
*****, **/**, **mod**, **rem**
unary -, **+**
binary +, **-**, **&**
relops, **in**, **not in**
and, **or**, **xor**, **and then**, **or else**
- C, C++, and Java have over 40 operators and at least 15 different levels of precedence

Short Circuit Evaluation

- An expression in which the result is determined without evaluating all of the operands and/or operators
- Example: **(13*a) * (b/13-1)**
If a is zero, there is no need to evaluate **(b/13-1)**
- Problem with non-short-circuit evaluation
index = 1;
while (index <= length) && (LIST[index] != value)
index++;
 - When **index=length**, **LIST [index]** will cause an indexing problem (assuming **LIST** has **length -1** elements)

Short Circuit Evaluation

- C, C++, and Java: use short-circuit evaluation for the usual Boolean operators (**&&** and **||**), but also provide bitwise Boolean operators that are not short circuit (**&** and **|**)
- Ada: programmer can specify either (short-circuit is specified with **and then** and **or else**)
- All logic operators in Ruby, Perl, ML, F#, and Python are short-circuit evaluated
- Short-circuit evaluation exposes the potential problem of side effects in expressions
e.g. **(a > b) || (b++ / 3)**

Assignment Statements

- The general syntax
`<target_var> <assign_operator> <expression>`
- The assignment operator
 - = FORTRAN, BASIC, the C-based languages
 - := ALGOL, Pascal, Ada
- = can be bad when it is overloaded for the relational operator for equality (that's why the C-based languages use == as the relational operator)

Assignment Statements: Conditional Targets

- Conditional targets (Perl)
`($flag ? $total : $subtotal) = 0`

Which is equivalent to

```
if ($flag){  
    $total = 0  
} else {  
    $subtotal = 0  
}
```

Assignment Statements: Compound Operators

- A shorthand method of specifying a commonly needed form of assignment
- Introduced in ALGOL; adopted by C
- Example:
 - $a = a + b$ is written as $a += b$

Assignment Statements: Unary Assignment Operators

- Unary assignment operators in C-based languages combine increment and decrement operations with assignment

- Examples

`sum = ++count` (count incremented, added to sum)

`sum = count++` (count incremented, added to sum)

`count++` (count incremented)

`-count++` (count incremented then negated)

Assignment as an Expression

- In C, C++, and Java, the assignment statement produces a result and can be used as operands
- An example:

```
while ((ch = getchar()) != EOF){...}
```

ch = getchar() is carried out; the result (assigned to ch) is used as a conditional value for the while statement

Disadvantage — side effect

List Assignments

- Perl and Ruby support list assignments

e.g.,

```
($first, $second, $third) = (20, 30, 40);
```

Assignment in Functional Languages

- Identifiers in functional languages are only names of values
- ML
 - Names are bound to values with **val**
val fruit = apples + oranges;
 - If another val for fruit follows, it is a new and different name
- F#
 - F#'s **let** is like ML's **val**, except **let** also creates a new scope

Mixed-Mode Assignment

- Assignment statements can also be mixed-mode
- In Fortran, C, and C++, any numeric type value can be assigned to any numeric type variable
- In Java, only widening assignment coercions are done
- In Ada, there is no assignment coercion